Gehad AlKady

Intermediate Importing Data in Python Notes

Importing data from the internet

- .txt
- .csv

Urllib package ⇒ interface for fetching data across the web

urlopen( ) → accepts URLs instead of file names

ex

```
from urllib.request import urlretrieve

url = '                    '

urlretrieve(url, 'try.csv')
```

Pandas → data frame

```
import pandas as pd

df = pd.read_csv
        ('try.csv', sep=';')

print(df.head())  # print first 5 rows

import matplotlib.pyplot as plt
```

Import dataset without save it locally

```
df = pd.read_csv(      ,      )
```

URL

separator

```
# plot first column of df
df.iloc[:,0].hist()
plt.xlabel('height')
plt.ylabel('weight')
plt.show()
```

Importing nonflat files from the web (excel spread sheet)

```
xls = pd.read_excel(url, None)
```

url

sheet name

xls is a dictionary include keys which is the sheets names

```
print(xls.keys())
```

```
dict_keys(['1700', '1900'])
```

※ print heat of first sheet

```
print(xls['1700'].head())
```

```
  country  1700
0
1
2
```

HTTP requests to import files from the web :

GET requests using urlib

```
from urllib.request import urlopen,
                           Request
url = "https://www.wikipedia.org/"

request = Request(url)
```

```
response = urlopen(request)

html = response.read()

    response.close()
```

GET requests using requests

```
import requests

url = "https://www.wikipedia.org/"

r = requests.get(url)

text = r.text  # return html as a string
```

r.text: text is an attribute

Scraping the web in python

Beautiful Soup are used to parse and extract structured data from HTML

```
from bs4 import BeautifulSoup
import requests
url = 'https://www.crummy.com/
    software/BeautifulSoup/'
r = requests.get(url)
html_doc = r.text
soup = BeautifulSoup(html_doc)
```

BeautifulSoup object

```
print(soup.prettify())
```

① title `print(soup.title)`

② text `print(soup.get_text())`

③ find(all) extract all hyper links in the html

```
for link in soup.find_all('a')
    print(link.get('href'))
```

element

note that : hyperlinks are defined by the HTML tag <a>

part 2 :

Interacting with APIs to import data from the web

① Introduction to APIs and JSONS

- API : Application Programming Interface

set of rules and protocols allows different software applications to communicate and exchange data with each other and trigger actions

- ex: enable developers to integrate exsisting functionalities like payment processing or data services into their own applications without rebuilding them.

- the standard form for transfering data through APIs is the JSON file format.

- JSON : Javascript Object Notation

- Realtime server-to_browser Communication

- JSON stored in python on a dict

loading JSONs in PYTHON

```
import json

with open('snakes.json', 'r')
        as json_file:

json_data = json.load(json_file)

type(json_data) → dict
```

Print all key value pairs to the console in a dictionary &

```
for key , value in json_data.items
    print (key + ': ' , value )
```

access all dictionary

```
for key , value in json_data.items
    print (key + ': ' , value [k] )
```

access the value associated with each key

json.load ( ) converts json into a python Dictionary

json.data.keys ( ) iterates over all keys in the dictionary

json_data[K] retrieves the corresponding value for each key

② APIs and Interacting with the world wide web

API is a bunch of code that allows two software programs to communicate with each other.

Connecting to an API in Python

```
import requests
url = 'http://www.omdbapi.com/
      ?t=hackers'
r = requests.get(url)
json_data = r.json()
for key, value in json_data
                    .items():
    print(key + ': ', value)
```

http request

title — name of film

query string

③ The Twitter API and Authentication

1_ REST API
2_ Streaming API (Public Streams)

3_ GET statuses/sample
4_ Fire hose

- Tweets returns as JSONs

Using Tweepy: Authentication

```
import tweepy, json

access_token = "..."
access_token_secret = "..."
consumer_key = "..."
consumer_secret = "..."
```

Using Tweepy: Stream tweets!!

⁂ create Streaming object

```
stream = tweepy.Stream(consumer_key, consumer_secret, access_token, access_token_secret)
```

⁂ this line filters Twitter streams to capture data by keywords:

```
stream.filter(track=['apples', 'oranges'])
```

Seaborn → Statistical data visualization library

Now, we are able to import data from a wide array of sources